

Righteous IT *Join the crusade!*

Linux Investigation (Part 3)

Posted on May 6, 2026May 4, 2026 by Hal Pomeranz

Please read Part 1 (<https://righteousit.com/2026/05/04/linux-investigation-part-1/>) and Part 2 (<https://righteousit.com/2026/05/05/linux-investigation-part-2/>) for additional background

We've actually done quite well reconstructing the major points of our intrusion scenario, but there are definitely some lingering questions. Let's see if we can dive deeper into some of these artifacts.

LD_PRELOAD Rootkit Or Not?

It would be nice to positively identify `libymv.so.3` as an LD_PRELOAD rootkit, rather than just referring to it as the "suspicious library". Fortunately we have a memory image and can use Volatility to extract the library. We can use `linux.elfs` to dump all the objects associated with a PID. We believe that the SSH daemon was explicitly restarted to pick up `libymv.so.3`, so we'll dump that process. As we saw in Part 2 (<https://righteousit.com/2026/05/05/linux-investigation-part-2/>), that PID is 937.

```
(venv) $ mkdir dump-elfs-pid-937
```

```
(venv) $ vol -q -f memory_dump/avml.lime -o dump-elfs-pid-937 linux.elfs -  
-pid 937 --dump
```

```
Volatility 3 Framework 2.27.1
```

PID	Process	Start	End	File Path	File Output
937	sshd	0x55e3f52a9000	0x55e3f52b2000	/usr/sbin/sshd	
pid.937.sshd.0x55e3f52a9000.dmp					
937	sshd	0x7f7da1093000	0x7f7da1098000	/usr/lib/x86_64-linux-gnu/libzstd.so.1.5.7	pid.937.sshd.0x7f7da1093000.dmp
937	sshd	0x7f7da115d000	0x7f7da1160000	/usr/lib/x86_64-linux-gnu/libpcre2-8.so.0.14.0	pid.937.sshd.0x7f7da115d000.dmp
937	sshd	0x7f7da120c000	0x7f7da1234000	/usr/lib/x86_64-linux-gnu/libc.so.6	pid.937.sshd.0x7f7da120c000.dmp
937	sshd	0x7f7da1400000	0x7f7da14f7000	/usr/lib/x86_64-linux-gnu/libcrypto.so.3	pid.937.sshd.0x7f7da1400000.dmp
937	sshd	0x7f7da1a3c000	0x7f7da1a3f000	/usr/lib/x86_64-linux-gnu/libz.so.1.3.1	pid.937.sshd.0x7f7da1a3c000.dmp
937	sshd	0x7f7da1a5e000	0x7f7da1a65000	/usr/lib/x86_64-linux-gnu/libselinux.so.1	pid.937.sshd.0x7f7da1a5e000.dmp
937	sshd	0x7f7da1aa5000	0x7f7da1aa7000	/usr/lib/x86_64-linux-gnu/libymv.so.3	pid.937.sshd.0x7f7da1aa5000.dmp
937	sshd	0x7f7da1ab3000	0x7f7da1ab5000	[vdso]	pid.937.sshd.0x7f7da1ab3000.dmp
937	sshd	0x7f7da1ab5000	0x7f7da1ab6000	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2	pid.937.sshd.0x7f7da1ab5000.dmp
937	sshd	0x7f7da1ade000	0x7f7da1ae9000	/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2	pid.937.sshd.0x7f7da1ade000.dmp

libymv.so.3 was extracted to dump-elfs-pid-937/pid.937.sshd.0x7f7da1aa5000.dmp. We could go full tilt at this and load it up for static analysis. But in the middle of an incident, I'm much more likely to do a first pass using "strings" and an internet search engine. Here are some of the more interesting strings I see in this sample

```
lpe_drop_shell  
timebomb  
backconnect  
Enjoy the shell!  
/tmp/silly.txt
```

Running those terms through my favorite search engine, my first hit was this useful [list of rootkit IOCs \(https://github.com/fkie-cad/linux-rootkit-iocs/blob/main/ioc-table.md\)](https://github.com/fkie-cad/linux-rootkit-iocs/blob/main/ioc-table.md). Apparently these strings are indicators for the [Father rootkit \(https://github.com/mav8557/Father\)](https://github.com/mav8557/Father).

If you look at the documentation for Father, it allows process, file, and directory hiding via a hard-coded GID. In our case this is apparently GID 7823 as we saw in [Part 2 \(https://righteousit.com/2026/05/05/linux-investigation-part-2/\)](https://righteousit.com/2026/05/05/linux-investigation-part-2/). Father also has an accept() hook backdoor which is activated by to connecting to any network service from a hard-coded port. Based on the linux.sockstat output, this appears to be port 48411 in our case.

Father also has a PAM hook that steals user passwords and saves them to `/tmp/silly.txt`. UAC captured this file for us:

```
(venv) $ cat \[root\]/tmp/silly.txt
100:1:password:worker
```

“worker” is the password for the `worker` account. The rest of the values here are hard-coded in the Father source code and are meaningless.

So even without static analysis I’m willing to state with high confidence that `libymv.so.3` is the Father LD_PRELOAD rootkit. And since I implanted it into the system, I can also state authoritatively that this is exactly what the library is.

Is top Really XMRig?

Let’s try a different approach to discover whether our suspicious “top” process is really XMRig as the command history we extracted in [Part 2 \(https://righteousit.com/2026/05/05/linux-investigation-part-2/\)](https://righteousit.com/2026/05/05/linux-investigation-part-2/) suggests. `linux.proc.MemoryMaps` allows us to dump the memory space of a process.

```
(venv) $ mkdir dump-mem-pid-977
(venv) $ vol -q -f memory_dump/avml.lime -o dump-mem-pid-977
linux.proc.MemoryMaps --pid 977 --dump
Volatility 3 Framework 2.27.1
```

PID	Process	Start	End	Flags	PgOff	Major	Minor	Inode
File Path		File	output					
977	top	0x400000		0x401000		r--	0x0	0
7	/dev/shm/kit/top (deleted)			pid.977.vma.0x400000-0x401000.dmp				26
977	top	0x401000		0xa5f000		r-x	0x1000	0
7	/dev/shm/kit/top (deleted)			pid.977.vma.0x401000-0xa5f000.dmp				26
977	top	0xa5f000		0xc40000		r--	0x65f000	0
26	7	/dev/shm/kit/top (deleted)		pid.977.vma.0xa5f000-0xc40000.dmp				0

[...]

```
(venv) $ grep -rLF xmrigh dump-mem-pid-977/
dump-mem-pid-977/pid.977.vma.0xa5f000-0xc40000.dmp
(venv) $ strings -a dump-mem-pid-977/pid.977.vma.0xa5f000-0xc40000.dmp
[...]
Usage: xmrigh [OPTIONS]
Network:
  -o, --url=URL          URL of mining server
  -a, --algo=ALGO        mining algorithm
https://xmrigh.com/docs/algorithms
  --coin=COIN            specify coin instead of algorithm
[...]
```

Finding the help text for the `xmrig` program in memory is conclusive enough for me. This is not a terribly surprising result, but it is nice to have additional confirmation.

Note that I tried a similar approach with `linux.proc.Maps` to try and recover the “`config`” file from the outbound SSH process (PID 975), but was unsuccessful. But while searching the memory strings for “`:3333`”, you can see some of the chatter from the XMRig process down the SSH tunnel.

```
[...]
4a747a5e140b7eb34933347c5154cb3d671bedc2e83b3a5e16f1603a4b660000
127.0.0.1:3333
method:"submit", "params":
{"id":"c1dcc2a9", "job_id":"77", "nonce":"ef000000", "result":"982e4063bea923
30be26b808c7d21d3fc349bc06f0c232fd673b7a827ab30000", "algo":"rx/0"}}
"cn/rto", "cn/rwz", "cn/zls", "cn/double", "cn/ccx", "cn-lite/1", "cn-
heavy/0", "cn-heavy/tube", "cn-heavy/xhv", "cn-pico", "cn-
pico/tlo", "cn/upx2", "rx/0", "rx/wow", "rx/arq", "rx/graft", "rx/sfx", "rx/yada"
, "argon2/chukwa", "argon2/chukwav2", "argon2/ninja", "ghostrider"]}]
[...]
```

Timeline Analysis

I’m passionate about timeline analysis, and will often use it in the early stages of a case to find indicators. However, we’ve been very successful with the other evidence sources that UAC provided and timeline analysis hasn’t been a priority. But now we can use timeline analysis to fill in any details we might have missed.

First we need to create the timeline from the body file provided by UAC with the help of the Sleuthkit’s `mactime` tool (“`mactime -d -y -b bodyfile/bodyfile.txt >bodyfile/timeline.csv`”). Alternatively, you could use my [ptt.sh](https://github.com/halpomeranz/dfis/blob/master/ptt.sh) (<https://github.com/halpomeranz/dfis/blob/master/ptt.sh>) script which creates a timeline that merges file system information with security log information including user logins, Sudo commands, etc.

After loading the timeline into our CSV viewer of choice, we can jump to 2026-03-24 23:22:19– the time of the “`worker`” login for the session where the Father rootkit was implanted. As usual, there is a lot of noise in the timeline, but the timeline generally confirms events we have discovered already.

Recall from [Part 1](https://righteousit.com/2026/05/04/linux-investigation-part-1/) (<https://righteousit.com/2026/05/04/linux-investigation-part-1/>) that the logs showed us a brief SSH session at 23:23:48. This session was not logged in `/var/log/wtmp`, indicating that it most likely was a single command or `scp` session that was not allocated a PTY and did not spawn an interactive shell. The timeline shows that at 23:23:48 the last access time on the “`/run/shm -> /dev/shm`” symlink was updated. Does this mean that the SSH connection at 23:23:48 was how the `/dev/shm/kit` directory was staged? This is certainly a plausible explanation, but not conclusive.

We know that the Father rootkit exfiltrates passwords in the file `/tmp/silly.txt`. The timeline shows us that this file was created at 23:34:32. This is when the SOC logged in as user `worker` to collect system data with UAC. This is just further confirmation that the Father rootkit was

operational on the system.

Wrapping Up

At this point, we have answers for the important questions for the scenario:

1. *Is the system compromised?* Definitely yes!
2. *How did the attackers gain access to the system?* They logged in as user “worker”, using the account password “worker”. This password was easily guessable, or it may have been disclosed or stolen. User “worker” had unlimited Sudo access, so privilege escalation was trivial.
3. *Why is there unencrypted traffic on port 22/tcp?* The attacker installed a rootkit that creates a backdoor in any networked service on the system, giving any user connecting from source port 48411/tcp a root shell on the system.
4. *What is consuming CPU on the system?* Process ID 975 is the XMRig cryptocurrency miner running under the executable name “top”. This process is connecting out through an SSH tunnel to 192.168.5.95. This system is now in scope and must be investigated.
5. *Why can't the SOC see what is happening on the system?* The rootkit installed by the attacker is hiding multiple processes from the attacker's backdoor session, including the cryptocurrency miner.

Here is our final timeline of important events during the incident:

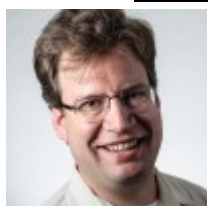
```
23:22:19    User "worker" logs in with password from jump host
(192.168.4.35) port 48364 [logs]
23:23:34    User "worker" uses sudo to execute root shell [logs]
23:23:48    Command-only/scp as user "worker" from jump host port 55504
[logs]
23:23:48    atime update on /dev/shm symlink, possible rootkit staging
[bodyfile]
23:24:51    Father rootkit installed as /usr/lib/x86_64-linux-
gnu/libymv.so.3 [bodyfile]
23:25:09    /etc/ld.so.preload created, points to .../libymv.so.3
[chkrootkit]
23:25:19    SSH daemon restarted [logs]
23:26:07    Unencrypted connect from 192.168.4.35 port 48411 via Father
rootkit hook [memory]
23:26:22    python3 execution to promote raw shell [memory]
23:26:22    Hidden bash process started from python pty.spawn() [memory]
23:27:16    User "worker" SSH session from jump host port 48364 ends
[logs]
23:27:51    /dev/shm/kit/xmrig renamed to "top" [memory]
23:28:17    ssh to 192.168.5.95 with tunnel on 3333/tcp [memory]
23:29:09    "top" process (renamed xmrig) started, comms via SSH tunnel
[memory]
23:29:19    /dev/shm/kit removed [memory]
23:34:32    SOC logs in to start collecting data with UAC [logs]
23:34:32    Father rootkit stores "worker" password in /tmp/silly.txt
[bodyfile]
```

Those of you who conducted your own investigation may have been thrown off by earlier artifacts left behind by my aborted attempts to create the scenario data. For example, there are login failures as user “worker” that could be construed as a brute force attack against this account, system crashes, and errors with `libymv.so.3`. My intention was that the scenario started with the login at 23:22:19, but kudos to those of you who found the earlier artifacts and invented plausible explanations for them.

Some submissions noted the fact that the kernel taint warning was triggered and speculated about a possible kernel rootkit. But only one submission actually ran down the source of the taint warning and realized it was due to the VirtualBox assistant and not enemy action. This is serious investigative dedication! Bravo!

One last part of this series is yet to come. I will be walking through what I actually did to create the scenario activity so that you can compare your answers with what really happened. In the meantime, don't forget to check out the reports from the [contest winners](#) (<https://www.linkedin.com/feed/update/urn:li:activity:7457038612812197890/>).

Posted in [DFIR](#), [Forensics](#), [Linux](#) Tagged [Contest](#), [DFIR](#), [Forensics](#), [Linux](#), [Scenario](#)



Published by Hal Pomeranz

Independent Computer Forensics and Information Security consultant. Expert Witness. Trainer. [View all posts by Hal Pomeranz](#)

1 thought on “Linux Investigation (Part 3)”

1. [Linux Investigation \(Part 4\) – Righteous IT](#) [May 7, 2026](#)

[...] In this installment I'm going to tell you the sequence of steps I took to create the scenario data. If you want to read about my investigation before you look at “the answers”, then please read Part 1, Part 2, and Part 3. [...]

[Blog at WordPress.com.](#)